

03 – CMake, PS & MenuConfig – Session Notes

CMake + Project Structure + Menuconfig

1. What Is a Build System?

When you write code in a file — say `main.c` — the computer cannot run it directly. It needs to be compiled into machine code first. On a small project with one file, you could compile manually:

```
gcc main.c -o my_program
```

But in a real project you have dozens or hundreds of files. You have libraries. You have different settings for debug vs release builds. You have multiple chips to target. Doing this manually is impossible.

A **build system** is a tool that knows:

- Which files need to be compiled
- In what order to compile them
- Which libraries to link
- What flags to use
- When to recompile (only changed files — not everything)

PC Analogy

Think of building a house. You have:

- Architects who draw the plans (your source code)
- A **project manager** who reads the plans and tells each worker what to do in what order (the build system — CMake)
- Workers who do the actual construction (the compiler — GCC/Clang)
- Tools the workers use (hammer, drill — compiler flags)

CMake is the project manager. You write the plans. CMake figures out the rest.

Build System Comparison

Tool	Used by	What it does
Arduino IDE	Arduino	Hides everything — one button compile
Make	Old C projects	Simple but hard to scale
CMake	ESP-IDF, Qt, Linux	Modern, cross-platform, scalable
Gradle	Android	Java/Kotlin projects
npm/webpack	Web	JavaScript projects

ESP-IDF uses CMake. Once you understand it — you understand how most professional C/C++ projects are built.

2. The ESP-IDF Project Structure

Every ESP-IDF project has the same folder structure. This is not optional — CMake and idf.py expect exactly this layout.

```
my_project/
|
+-- CMakeLists.txt          <- Top-level build file (Project level)
+-- sdkconfig               <- Generated config (never edit this)
+-- sdkconfig.defaults      <- Your config overrides (edit this)
+-- dependencies.lock       <- Component versions (auto-generated)
|
+-- main/                  <- Your main application component
|   +-- CMakeLists.txt     <- Main component build file
|   +-- main.c             <- Entry point — app_main() lives here
|   +-- main.h             <- Optional header
|
+-- components/            <- Your custom reusable components
|   +-- my_sensor/
|       +-- CMakeLists.txt
|       +-- my_sensor.h
|       +-- my_sensor.c
|
+-- build/                 <- Generated output (never commit this)
    +-- my_project.bin     <- Final firmware binary
    +-- bootloader/
    +-- partition_table/
```

3. CMakeLists.txt — The Two Files

Every ESP-IDF project has **two** CMakeLists.txt files. This confuses beginners. Here is exactly what each one does.

File 1 — Root CMakeLists.txt

This is the top-level project definition. It tells CMake:

- What this project is called
- Where to find ESP-IDF
- What the minimum CMake version is

```
cmake_minimum_required(VERSION 3.16)

include($ENV{IDF_PATH}/tools/cmake/project.cmake)
```

```
project(my_project)
```

That is it. Three lines. You almost never change this file.

Line	What it does
<code>cmake_minimum_required</code>	CMake version check — ensures compatibility
<code>include(...)</code>	Loads all ESP-IDF build rules and toolchain
<code>project(my_project)</code>	Names your project — appears in build output

File 2 — main/CMakeLists.txt

This is the component-level definition. It tells CMake:

- Which source files belong to this component
- Which other components this component needs

```
idf_component_register(  
    SRCS "main.c"  
    INCLUDE_DIRS "."  
)
```

Line	What it does
<code>idf_component_register</code>	Registers this folder as an ESP-IDF component
<code>SRCS "main.c"</code>	List of source files to compile
<code>INCLUDE_DIRS "."</code>	Folders where header files are found

Adding More Source Files

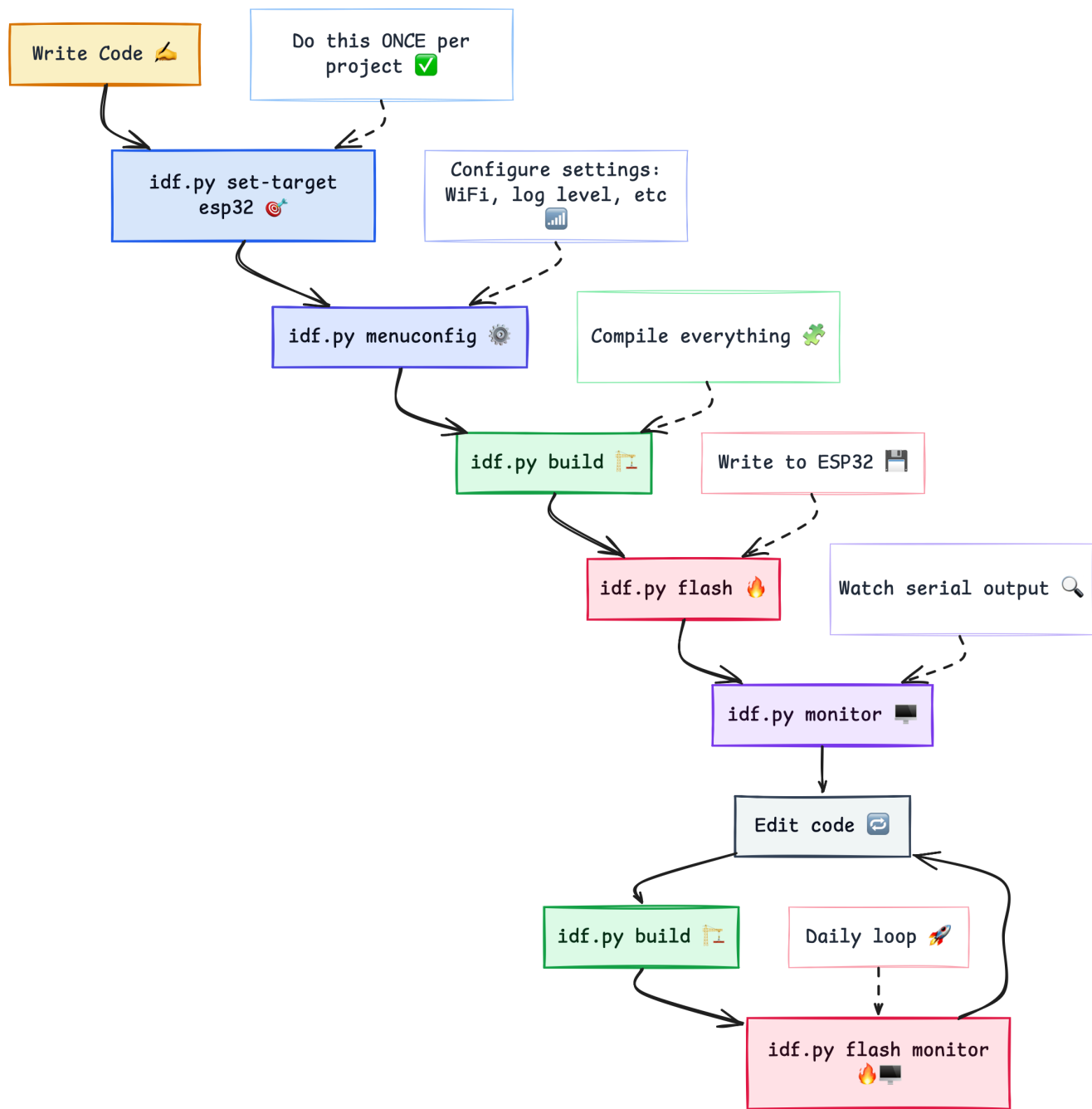
```
idf_component_register(  
    SRCS  
        "main.c"  
        "wifi_manager.c"  
        "sensor_reader.c"  
    INCLUDE_DIRS "."  
    REQUIRES  
        nvs_flash  
        esp_wifi  
        driver  
)
```

REQUIRES lists which ESP-IDF components your code depends on. CMake uses this to pull in the right libraries automatically.

4. The idf.py Workflow

idf.py is your daily command. It wraps CMake and provides a simple interface.

The Full Workflow



Command Reference Table

Command	What it does	When to use
idf.py create-project NAME	Creates new project folder	Once at start
idf.py set-target esp32	Sets target chip	Once per project
idf.py menuconfig	Opens config UI	When changing settings
idf.py build	Compiles project	After every code change
idf.py flash	Flashes to board	After build

Command	What it does	When to use
<code>idf.py monitor</code>	Opens serial monitor	To see output
<code>idf.py flash monitor</code>	Flash then monitor	Most common combo
<code>idf.py fullclean</code>	Deletes build folder	When build is broken
<code>idf.py size</code>	Shows binary size	When debugging memory
<code>idf.py -p COM3 flash</code>	Flash to specific port	Windows
<code>idf.py -p /dev/ttyUSB0 flash</code>	Flash to specific port	Linux/Mac

Analogy for Build Process

Your source code (.c files) 

idf.py build 

CMake reads
CMakeLists.txt 

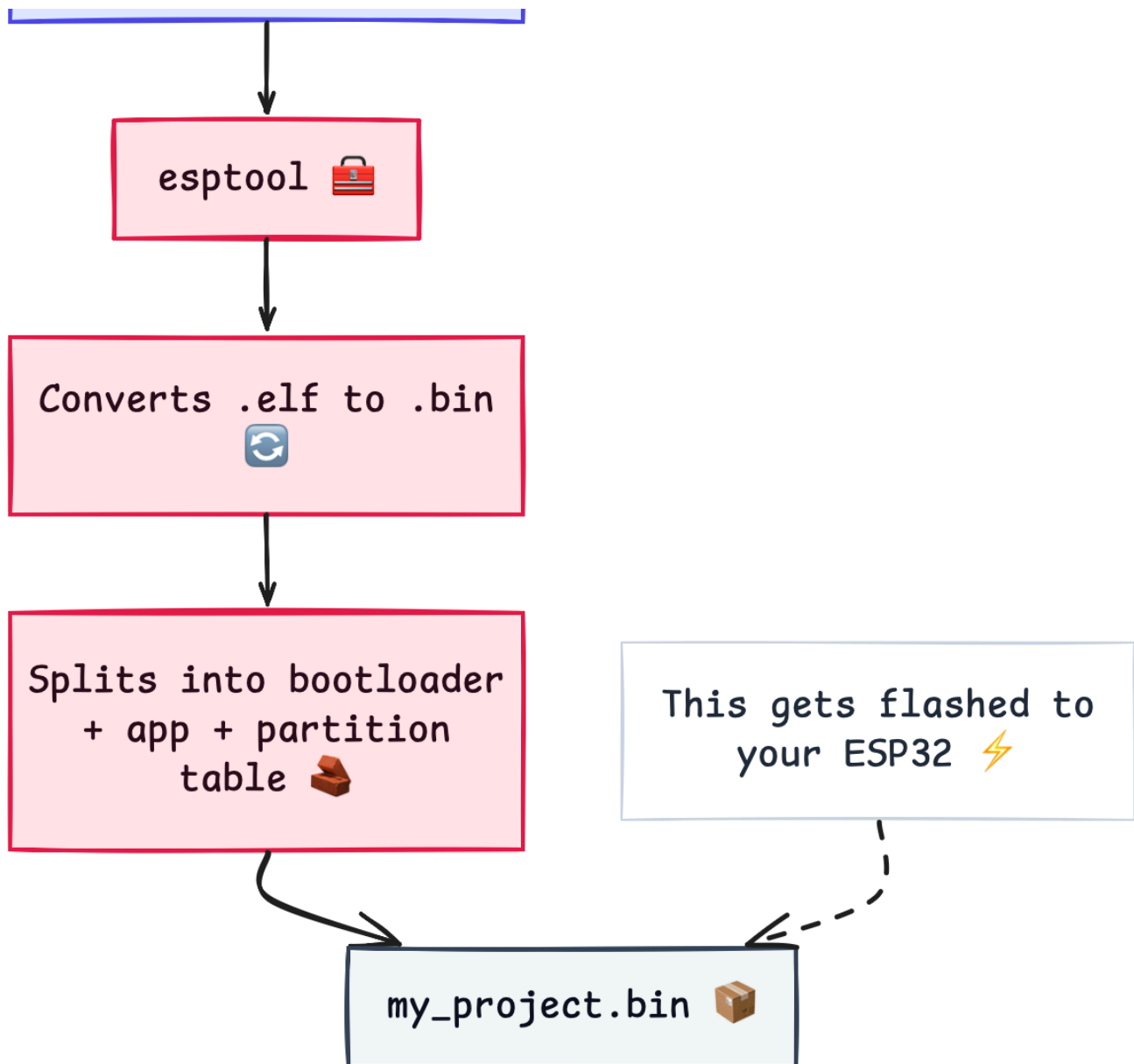
Figures out what to
compile 

GCC Compiler 

Turns .c files into .o
object files 

Linker 

Combines all .o files
+ libraries into one
.elf file 



This is exactly how your PC's software is built — just targeting different hardware.

5. What `idf.py set-target` Does

This is one of the most important commands — and most misunderstood.

```
idf.py set-target esp32
idf.py set-target esp32s3
idf.py set-target esp32c3
```

When you run this, ESP-IDF:

1. Sets the correct compiler toolchain (Xtensa GCC for ESP32/S3, RISC-V GCC for C3/C6)
2. Sets the correct memory layout for that chip
3. Sets the correct peripheral register addresses
4. Generates a base `sdkconfig` for that chip
5. Clears any previous build output

Why This Matters

```
ESP32      uses Xtensa LX6  -> xtensa-esp32-elf-gcc compiler
ESP32-S3    uses Xtensa LX7  -> xtensa-esp32s3-elf-gcc compiler
ESP32-C3    uses RISC-V      -> riscv32-esp-elf-gcc compiler
```

The same C code — different binary output for each chip. `set-target` switches which compiler and which chip config to use.

Rule: Run `idf.py set-target` once when creating a project. If you change the target — run `idf.py fullclean` first to wipe the old build.

6. Menuconfig — Configuring Your Project

Menuconfig is a graphical configuration system built into ESP-IDF. It lets you configure hundreds of project settings without editing any code.

```
idf.py menuconfig
```

This opens an interactive menu in your terminal:

```
ESP-IDF Configuration
|
+-- Component config
|  |
|  +-- FreeRTOS
|  |   +-- Kernel tick frequency (Hz)          [1000]
|  |   +-- Maximum task priorities              [25]
|  |   +-- Enable task watchdog                [yes]
|  |
|  +-- WiFi
|  |   +-- WiFi static RX buffer number        [10]
|  |   +-- WiFi NVS flash                      [yes]
|  |   +-- WiFi task core ID                   [0]
|  |
|  +-- Log output
|  |   +-- Default log verbosity                [Info]
|
+-- Application configuration <- YOUR custom settings go here
    +-- WiFi SSID                             [your_network]
    +-- WiFi Password                         [your_password]
    +-- MQTT Broker URL                       [mqtt://broker:1883]
```

PC Analogy

Menuconfig is like the **Windows Control Panel** or **BIOS settings** for your firmware.

Menuconfig setting	PC equivalent
FreeRTOS tick rate	Windows scheduler frequency
Log verbosity	Windows Event Log level
WiFi buffer size	Network adapter buffer size
Flash encryption	BitLocker encryption
Bootloader log level	BIOS verbose mode
Task watchdog timeout	Windows hang detection timeout

Navigation Keys in Menuconfig

Key	Action
Arrow keys	Move up/down
Enter	Open submenu / select
Space	Toggle yes/no options
Esc	Go back one level
/	Search for a setting
?	Show help for current item
S	Save configuration
Q	Quit

7. sdkconfig — What Gets Generated

After you run menuconfig and save, ESP-IDF generates a file called `sdkconfig`.

```
# Automatically generated file – DO NOT EDIT
CONFIG_IDF_TARGET="esp32"
CONFIG_FREERTOS_HZ=1000
CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ_240=y
CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM=10
CONFIG_LOG_DEFAULT_LEVEL_INFO=y
CONFIG_LOG_DEFAULT_LEVEL=3
...
(hundreds more lines)
```

The Golden Rule

Never manually edit `sdkconfig`.

It is auto-generated. Any manual changes get overwritten next time you run menuconfig. If you need to set a default value — use `sdkconfig.defaults` .

sdkconfig.defaults

This is the file you control. Put only the settings you want to override:

```
# sdkconfig.defaults
CONFIG_FREERTOS_HZ=1000
CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ=240
CONFIG_LOG_DEFAULT_LEVEL_WARN=y
CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM=16
```

ESP-IDF applies these defaults every time someone runs `idf.py menuconfig` on a fresh clone of your repo. This is how you share consistent config across a team.

PC Analogy

File	PC Equivalent
<code>sdkconfig</code>	Windows Registry — auto-generated, system managed
<code>sdkconfig.defaults</code>	Group Policy — your organisation's baseline settings

8. Components — Splitting Your Code

As your project grows, putting everything in `main.c` becomes unmanageable. ESP-IDF solves this with **components** — self-contained code modules.

What a Component Is

A component is a folder with:

- Its own `CMakeLists.txt`
- One or more `.c` source files
- One or more `.h` header files

```
components/
+-- wifi_manager/
|   +-- CMakeLists.txt
|   +-- wifi_manager.h      <- Public API — what others can use
|   +-- wifi_manager.c      <- Implementation — private details
|
+-- sensor_reader/
    +-- CMakeLists.txt
    +-- sensor_reader.h
    +-- sensor_reader.c
```

Component CMakeLists.txt

```
idf_component_register(
    SRCS "wifi_manager.c"
    INCLUDE_DIRS "."
    REQUIRES
        esp_wifi
        nvs_flash
        esp_event
)
```

Using a Component in main.c

```
#include "wifi_manager.h" // Include the component header
#include "sensor_reader.h"

void app_main(void) {
    wifi_manager_init();
    wifi_manager_connect("MySSID", "MyPassword");

    sensor_reader_init();
    float temp = sensor_reader_get_temperature();
}
```

main/CMakeLists.txt — Declaring the Dependency

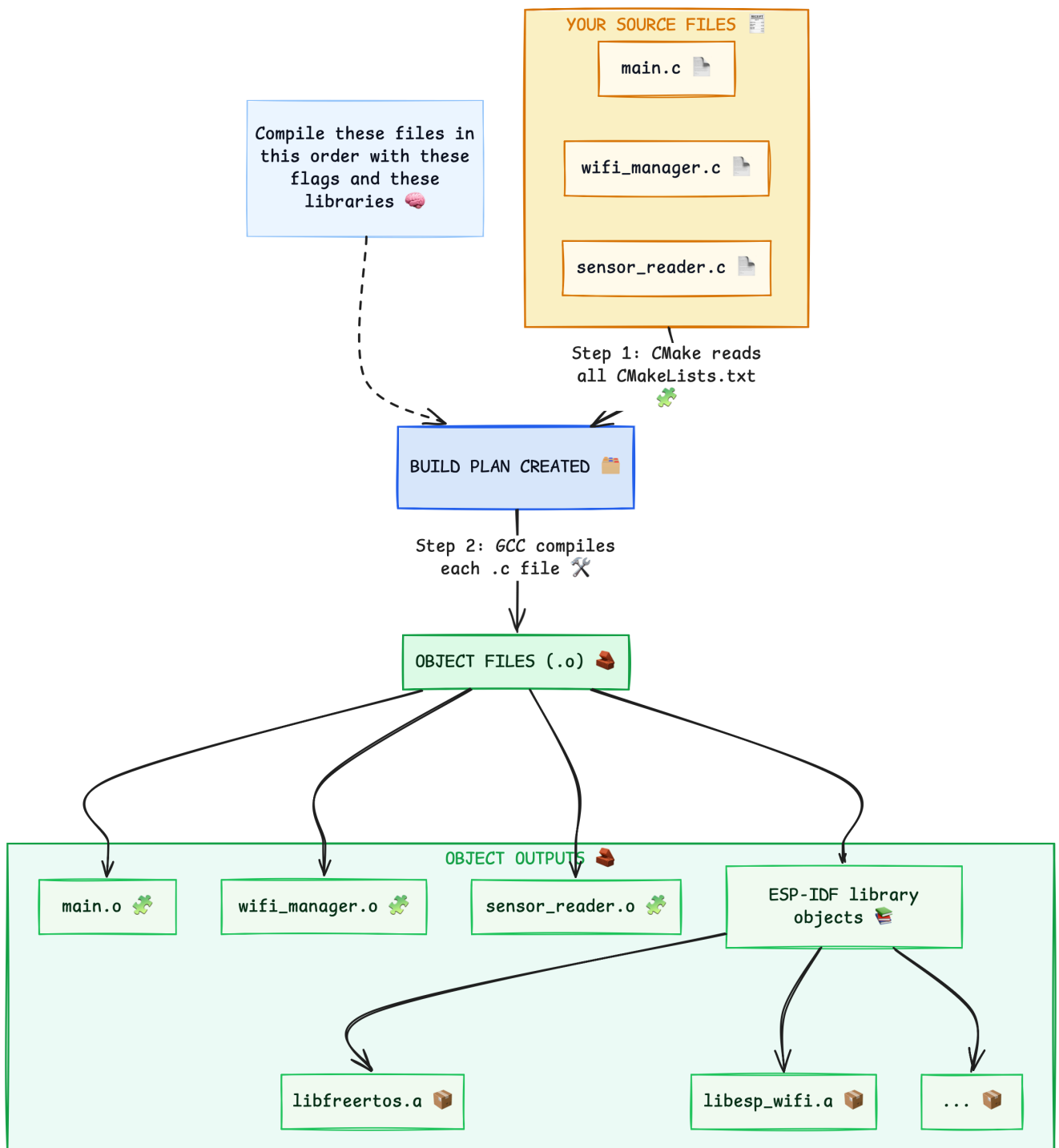
```
idf_component_register(
    SRCS "main.c"
    INCLUDE_DIRS "."
    REQUIRES
        wifi_manager      <- Your custom component
        sensor_reader      <- Your custom component
)
```

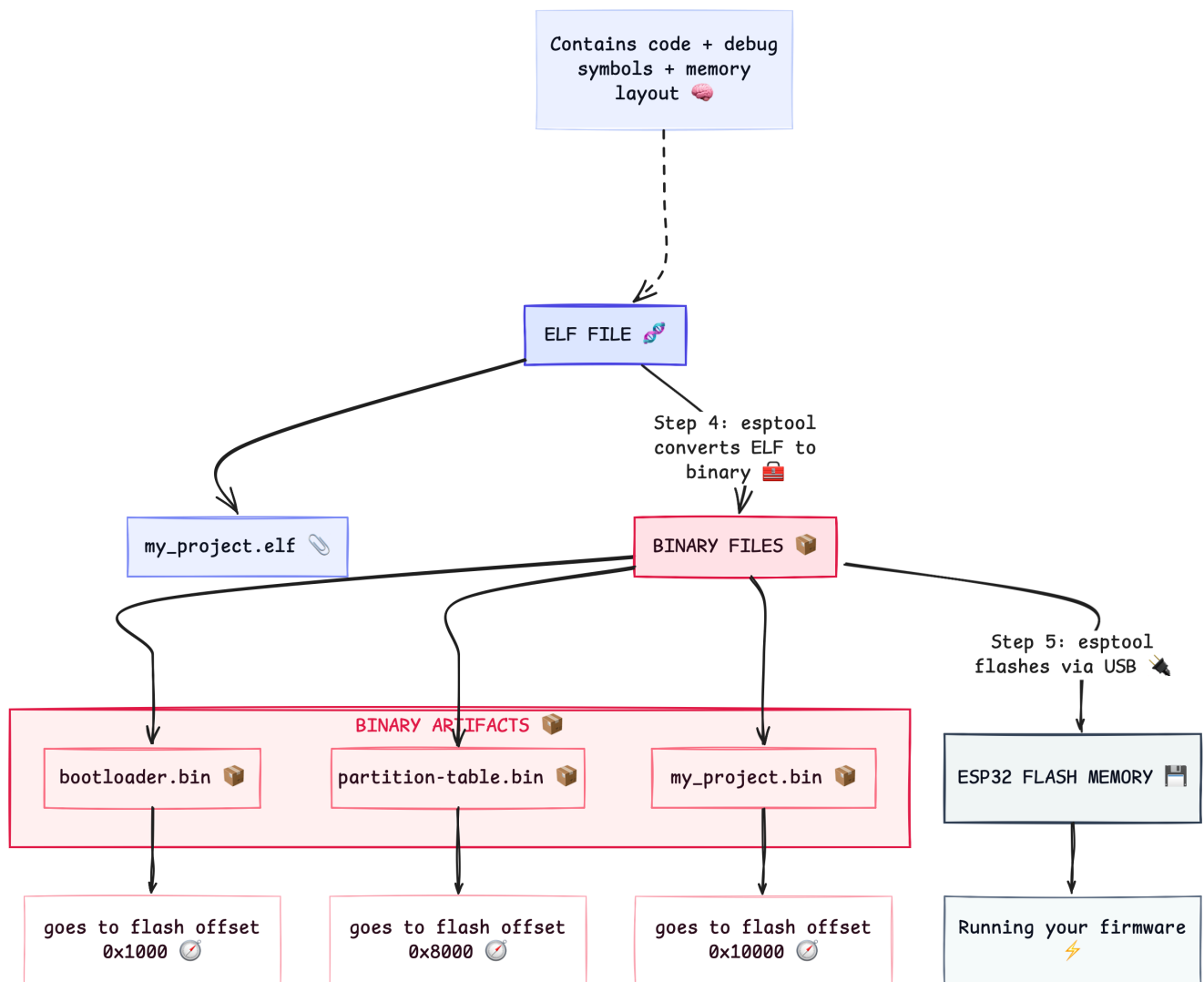
PC Analogy

Concept	PC Equivalent
Component	A DLL / shared library
wifi_manager.h	The DLL's public API header
wifi_manager.c	The DLL's internal implementation
REQUIRES in CMakeLists	Import / linker dependency declaration

9. The Complete Build Pipeline

Here is everything that happens from source code to flashed firmware:





10. Practical Example — Creating a Project from Scratch

Follow these steps exactly:

Step 1 — Activate ESP-IDF

```
# Windows — open IDF Terminal from EIM shortcut
# macOS/Linux
get_idf
```

Step 2 — Create the Project

```
mkdir -p ~/esp/workshop
cd ~/esp/workshop
idf.py create-project sensor_node
cd sensor_node
```

Step 3 — Set Target Chip

```
# For ESP32 DevKit
idf.py set-target esp32

# For XIAO ESP32-S3
idf.py set-target esp32s3

# For ESP32-C3 SuperMini
idf.py set-target esp32c3
```

Step 4 — Open and Explore Menuconfig

```
idf.py menuconfig
```

Navigate to:

- Component config → Log output → Default log verbosity → change to **Debug**
- Save with **S** → Quit with **Q**

Step 5 — Look at Generated sdkconfig

```
# On macOS/Linux
head -20 sdkconfig

# On Windows PowerShell
Get-Content sdkconfig | Select-Object -First 20
```

Step 6 — Write app_main

Open `main/main.c` in VS Code and replace contents with:

```
#include <stdio.h>
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

static const char *TAG = "workshop";

void app_main(void) {
    ESP_LOGI(TAG, "ESP32 Production Firmware Workshop");
    ESP_LOGI(TAG, "Session 03 - CMake and Menuconfig");

    int counter = 0;
    while (1) {
        ESP_LOGI(TAG, "Running... count = %d", counter++);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

Step 7 — Build

```
idf.py build
```

Look for at the end:

```
Project build complete.
```

This confirms your build system, project structure and toolchain are all working.

Summary

Concept	What it is	PC Equivalent
CMake	Build system	Visual Studio project manager
CMakeLists.txt (root)	Project definition	.sln solution file
CMakeLists.txt (main/)	Component definition	.csproj project file
idf.py	Build tool wrapper	MSBuild / dotnet CLI
set-target	Choose chip	Choose build platform (x86/ARM)
menuconfig	Config UI	Windows Control Panel / BIOS
sdkconfig	Generated config file	Windows Registry
sdkconfig.defaults	Your config overrides	Group Policy baseline
Component	Reusable code module	DLL / shared library
build/ folder	Compiled output	bin/ or dist/ folder
idf.py fullclean	Delete build output	Delete bin/ and rebuild